

ASYMPTOTIC NOTATIONS

DEFINITION:

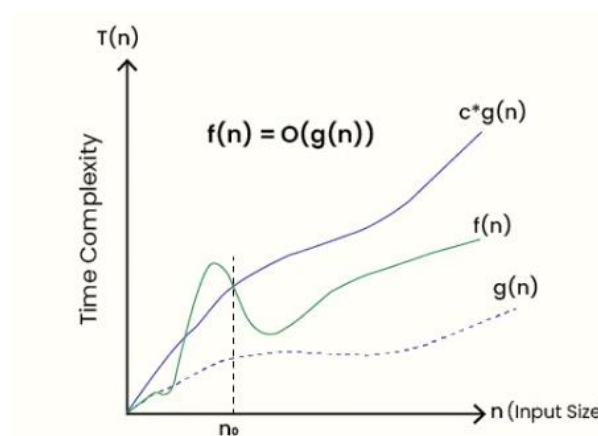
- Asymptotic Notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as the input size grows.
- These notations provide a concise way to express the behavior of an algorithm's time or space complexity as the input size approaches infinity.
- It enables comparisons of algorithms' efficiency by abstracting away machine-specific constants and implementation details, focusing instead on fundamental trends.
- Asymptotic analysis allows for the comparison of algorithms' space and time complexities by examining their performance characteristics as the input size varies.

TYPES OF ASYMPTOTIC NOTATIONS:

1. Big-O Notation (O-notation)
2. Omega Notation (Ω -notation)
3. Theta Notation (Θ -notation)

Big-O Notation (O-notation):

Given two functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$, if there exist constants $c > 0$ and $n_0 \geq 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.



Properties of Big O Notation:

1. Reflexivity:

For any function $f(n)$, $f(n) = O(f(n))$.

2. Transitivity:

If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

3. Constant Factor:

For any constant $c > 0$ and functions $f(n)$ and $g(n)$, if $f(n) = O(g(n))$, then $cf(n) = O(g(n))$.

4. Sum Rule:

If $f(n) = O(g(n))$ and $h(n) = O(k(n))$, then $f(n) + h(n) = O(\max(g(n), k(n)))$. When combining complexities, only the largest term dominates.

5. Product Rule:

If $f(n) = O(g(n))$ and $h(n) = O(k(n))$, then $f(n) * h(n) = O(g(n) * k(n))$.

6. Composition Rule:

If $f(n) = O(g(n))$, then $f(h(n)) = O(g(h(n)))$.

Type	Notation	Example
Logarithmic	$O(\log n)$	Binary Search
Linear	$O(n)$	Linear Search
Super linear	$O(n \log n)$	Heap Sort, Merge Sort

Worst, Average and Best Case Analysis of Algorithms:

i) WORST CASE ANALYSIS:

Worst-case complexity gives the upper limit on the time taken by the algorithm to execute. It is the maximum time needed for the algorithm to execute.

Linear Search Example:

In the worst-case scenario, the item to be searched either does not exist in the list or is positioned in the last index of the array. The algorithm will have to compare the item with all elements in the array.

Time Complexity: $O(n)$

Example:

Array = [10, 20, 30, 40, 50]

Item to search = 60

Number of Comparisons: 5

II) BEST CASE ANALYSIS

Best case analysis gives the minimum possible time taken by an algorithm for execution. It tells about the lower limit on the running time of the algorithm.

Example of Linear Search:

Best case arises when the search key is located in the first place. One comparison is enough.

Time complexity: $O(1)$

Illustration:

List = [10, 20, 30, 40, 50]

Search Item = 10

Number of Comparisons: 1

iii) Average Case Analysis:

Average case analysis computes the average cost of the execution of the algorithm, taking into account all the cases and their probability.

Linear Search Algorithm Example:

i) The search key could occur at any place in the array, or it might not be there at all. The average number of items examined until reaching the searched item is roughly half.

ii) Time Complexity: $O(n)$

Example:

Array = [10, 20, 30, 40, 50]

Search Key = 30

Average comparisons: $\sim n/2$

MOST COMMONLY USED:

Most commonly Worst case is used because for the multiple runtime input the search key can be present anywhere in the array.

